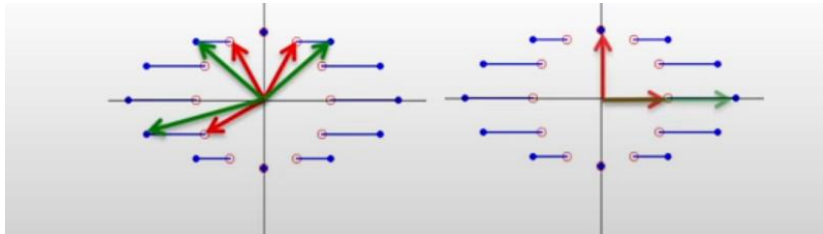
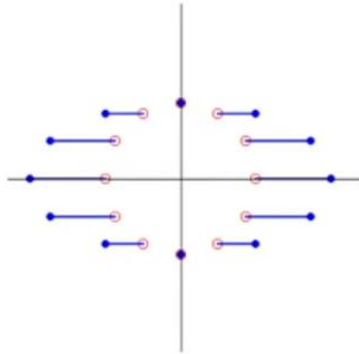
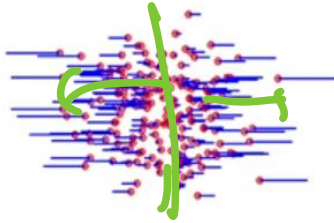


Principal Component Analysis

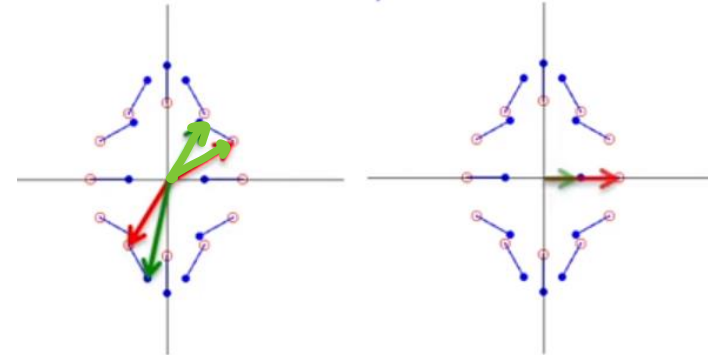
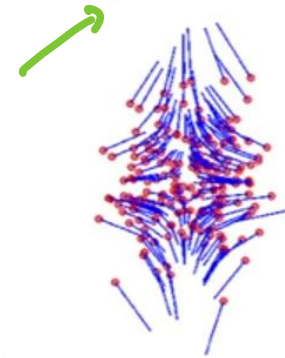
Topic 11

- Matrix transformations have special directions

$$A = \begin{pmatrix} 1+\delta & 0 \\ 0 & 1 \end{pmatrix} \begin{matrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \rightarrow \\ 2 \times 2 \quad 2 \times 1 \end{matrix}$$



$$A = \begin{pmatrix} 1-\delta & 0 \\ 0 & 1+\delta \end{pmatrix}$$



For a diagonal matrix, vectors along the axes are scaled but not rotated and the scaling factors are simply those on the diagonal

$$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \lambda_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$\downarrow$
 eval

$\hookrightarrow$ e. vector

$$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \lambda_2 \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \lambda \hat{e}_i = \lambda_i \hat{e}_i$$

Diagonal matrix $\Rightarrow$ (Standard basis vectors are the Eigen vectors)

Any vector $\vec{v}$ fixed is mapped by matrix A to another vector $\lambda \vec{v}$ is called an eigen vector, of $\vec{v}$. The scale factor λ is called eigen value.

$\mathbb{R}^n \Rightarrow n$ eigen vectors

Diag

$$\lambda \approx \begin{pmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{pmatrix} \Rightarrow \lambda \hat{e}_i = \lambda_i \hat{e}_i$$

$i = (1 \dots n)$

Standard Basis vector $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Any vector $\vec{v}$ that is mapped by matrix A onto another 11th vector $\lambda\vec{v}$ is called an eigenvector of A . The scale factor λ is called the eigenvalue of vector $\vec{v}$. $\mathbb{R}^n \Rightarrow$ n eigen vectors

Diag. matrix:

$$\left\{ \begin{array}{l} \lambda = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix} \\ \text{Standard basis vector} \Rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{array} \right\} \Rightarrow \left(\lambda \hat{e}_i = \lambda_i \hat{e}_i \right)_{i=1 \dots n}$$

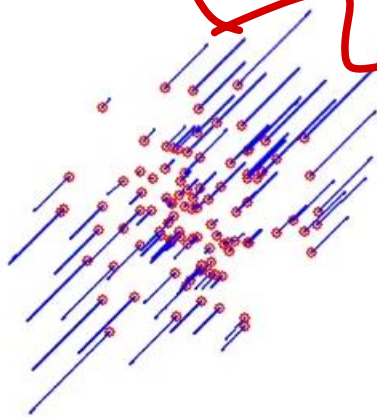
Rotated and stretched matrix

$$\underline{A} = \Phi \Lambda \Phi^T = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}$$

$$\Phi \lambda \Phi^T$$

$$\Lambda = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

Σ value matrix

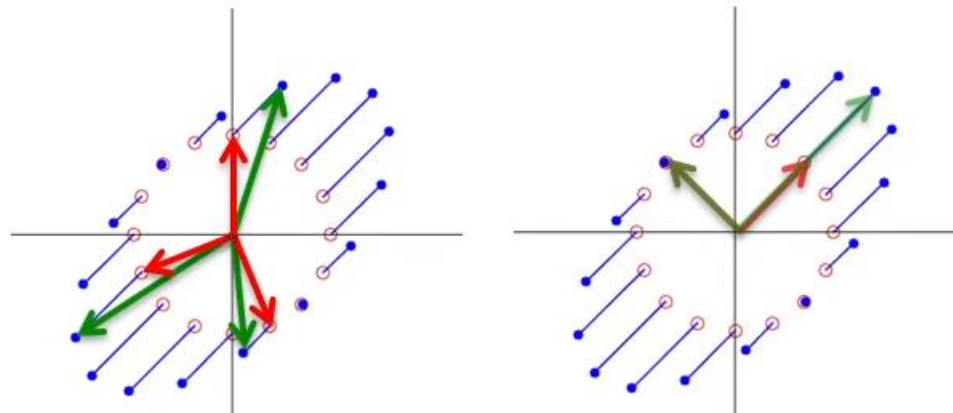


symmetric
Eigen vectors

$$\Phi(45^\circ) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

Rotation matrix

Eigen vectors



$$A \vec{n}_i = a_i \vec{n}_i$$

↓

$$\Phi \lambda \Phi^T \vec{x}_i = a_i \vec{n}_i$$

$$\underbrace{\Phi^T \Phi}_{I} \lambda \underbrace{\Phi^T \vec{x}_i}_{\text{diagonal matrix}} = \Phi^T a_i \vec{n}_i$$

↪ diagonal matrix

$$\underbrace{\lambda \Phi^T \vec{x}_i}_{\text{diag matrix}} = a_i \underline{\underline{\Phi^T \vec{x}_i}}$$

↪ diag matrix

$$A = \Phi \lambda \Phi^T$$

$$\Phi^T \vec{x}_i = \hat{e}_i$$

$$\vec{x}_i = \Phi \hat{e}_i$$

↓

eigen vector for $\textcircled{A}$

Eigen vector are just the standard basis vectors rotated by matrix Φ $\vec{x}_i = \Phi \hat{e}_i$

eigen value problem

$$A \vec{x} = a_i \vec{x}_i$$

↓

$$\phi \lambda \phi^T \vec{x}_i = a_i \vec{x}_i$$

$$\phi^T \phi \lambda \phi^T \vec{x}_i = \phi^T a_i \vec{x}_i$$

↓
I

$$\lambda \phi^T \vec{x}_i = a_i \phi^T \vec{x}_i$$

λ → diagonal matrix

$a_i \Rightarrow$
(eigenvalues)

$$\phi^T \vec{x}_i = \hat{e}_i$$

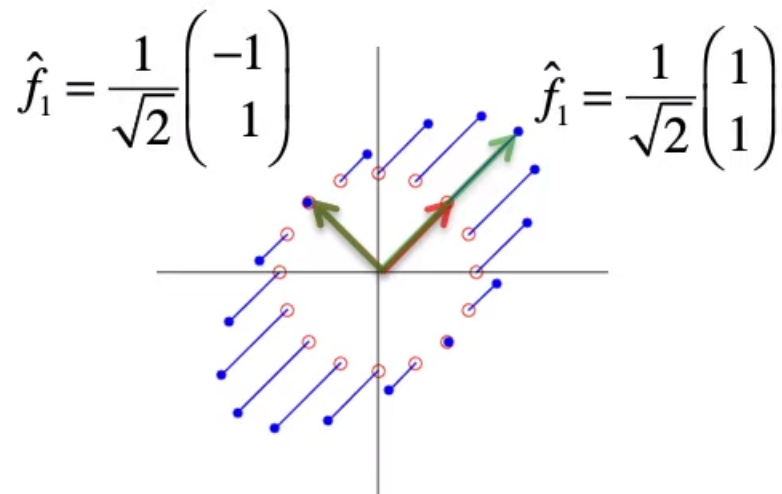
↓

$$\vec{x}_i = \underline{\underline{\phi \hat{e}_i}}$$

↓
eigen vector for A

- The eigenvectors are just the standard basis vectors rotated by the matrix Φ !

$$\vec{x}_i = \Phi \hat{e}_i$$



$$\Phi(45^\circ) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

$$\vec{x}_1 = \Phi \hat{e}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\vec{x}_2 = \Phi \hat{e}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

The eigenvectors are just the columns of Φ !

- In summary, a symmetric matrix A can always be written as follows:

$$A = \Phi \Lambda \Phi^T$$

where Φ is a rotation matrix and Λ is a diagonal matrix

- The eigenvectors of A are the columns of Φ (the basis vectors, $\hat{f}_i$)

$$\Phi = \left(\hat{f}_1 \mid \hat{f}_2 \right)$$

- The eigenvalue associated with each eigenvector $\hat{f}_i$ is the diagonal element λ_i of Λ .

$$\Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$$

- Note that eigenvectors are not unique...

If $\vec{x}_i$ is an eigenvector of A then so is $a\vec{x}_i$

$$A\vec{x}_i = \lambda_i \vec{x}_i$$

$$A(a\vec{x}_i) = \lambda_i (a\vec{x}_i)$$

... so we usually write eigenvectors as unit vectors

- For a matrix in n-dimensions... there are n different unit eigenvectors
- For a symmetric matrix, the eigenvectors of A are orthogonal (and we write them as unit vectors)...

... the eigenvectors of A form a complete orthonormal basis set!

Read:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A\vec{x} = \lambda I \vec{x}$$

$$(A - \lambda I) \vec{x} = 0$$

characteristic eqn

find λ where $x \neq 0$

$$A - \lambda I = \begin{pmatrix} a - \lambda & b \\ c & d - \lambda \end{pmatrix}$$

$$ad - \lambda(a+d) + \lambda^2 - bc = 0$$

$$\lambda^2 - (a+d)\lambda + ad - bc = 0$$

$$\lambda = \frac{1}{2}(a+d) \pm \frac{1}{2} \sqrt{(a-d)^2 + 4bc}$$

roots:
Complex
real / imag ^{parts}

$$A = \begin{bmatrix} a & b \\ b & d \end{bmatrix} \sim \begin{matrix} \text{eigenvalues of symm matrix} \\ \text{are always } \underline{\text{Real}} \end{matrix}$$

$$A = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix} \quad \lambda = 1 \pm \frac{\sqrt{2}}{2}$$

$$A = \begin{pmatrix} a & b \\ b & a \end{pmatrix} \quad \begin{array}{l} \lambda_1 = a + b \\ \hline \lambda_2 = a - b \\ \hline \end{array}$$

- Let's consider a special case of a symmetric matrix

$$A = \begin{pmatrix} a & b \\ b & a \end{pmatrix}$$

$$A\vec{x}_+ = \lambda_+ \vec{x}_+$$

$$\begin{pmatrix} a & b \\ b & a \end{pmatrix} \vec{x}_+ = (a+b) \vec{x}_+$$

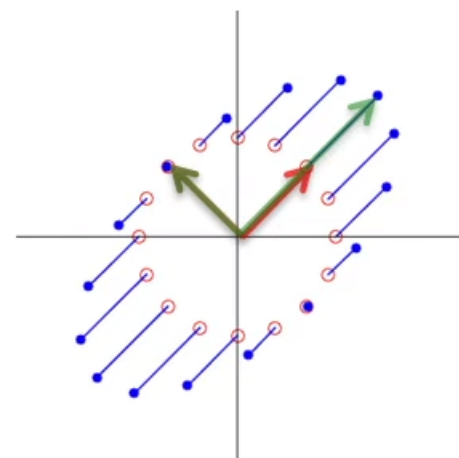
$$\begin{pmatrix} a & b \\ b & a \end{pmatrix} \vec{x}_+ = \begin{pmatrix} a+b & 0 \\ 0 & a+b \end{pmatrix} \vec{x}_+$$

$$\begin{pmatrix} -b & b \\ b & -b \end{pmatrix} \vec{x}_+ = 0$$

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0 \quad \begin{matrix} x_1 = s \\ x_2 = s \end{matrix}$$

$$\lambda_{\pm} = \frac{1}{2}(a+d) \pm \frac{1}{2}\sqrt{(a-d)^2 + 4b^2}$$

$$\lambda_+ = a+b \quad \lambda_- = a-b$$



$$\vec{x}_+ = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \vec{x}_- = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

Eigen-decomposition

- The process of writing a matrix A as $A = \Phi \Lambda \Phi^T$ is called eigen-decomposition. It works for any symmetric matrix.
 - the eigenvalues λ_i are real numbers
 - the eigenvectors $\hat{f}_i$ form an orthogonal basis set

- Let's rewrite...

$$A = \Phi \Lambda \Phi^T$$

$$\Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$$

$$A\Phi = \Phi \Lambda \Phi^T \Phi$$

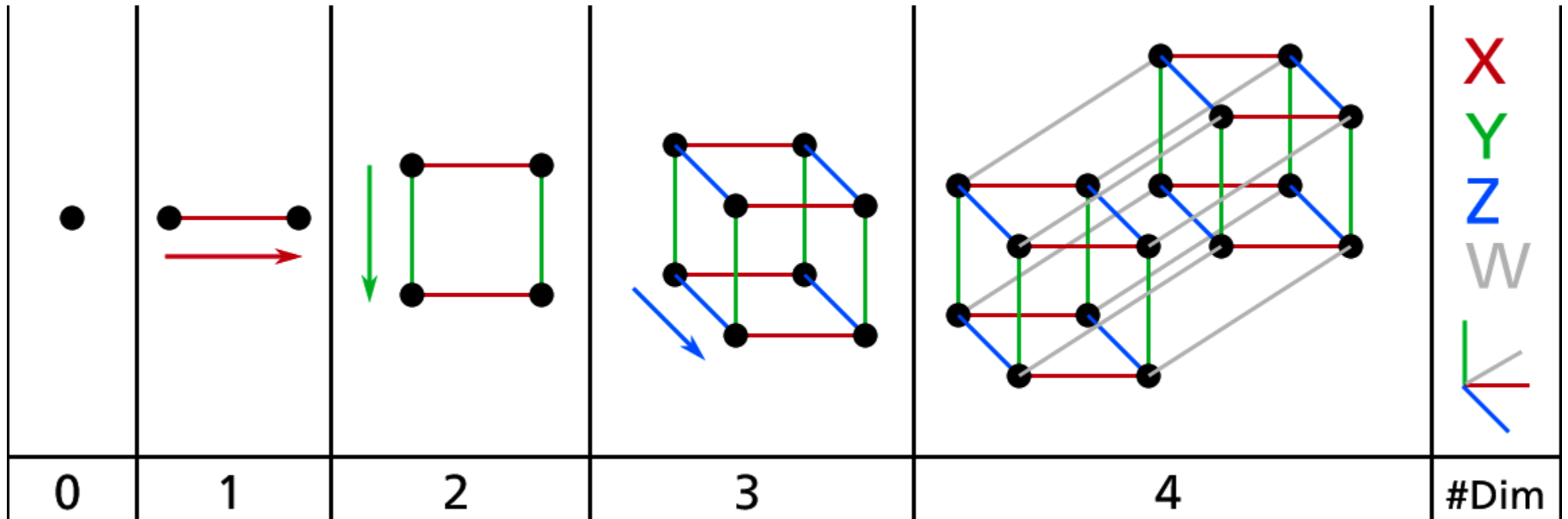
- Eigenvalue equation

$$A\Phi = \Phi \Lambda$$

is equivalent to the set of equations

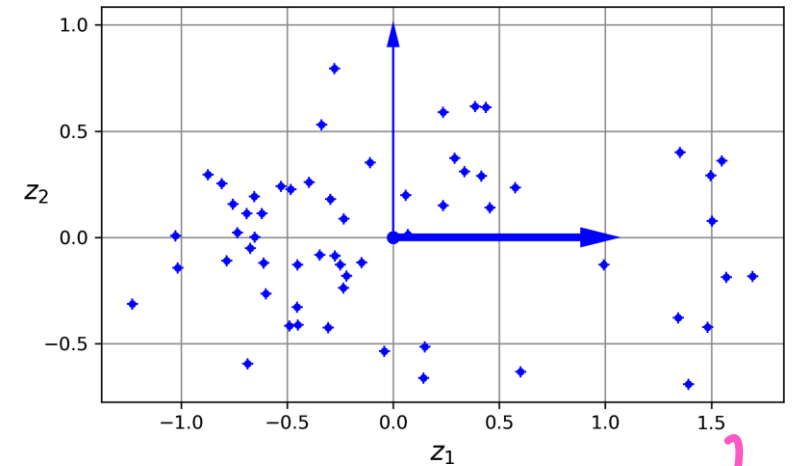
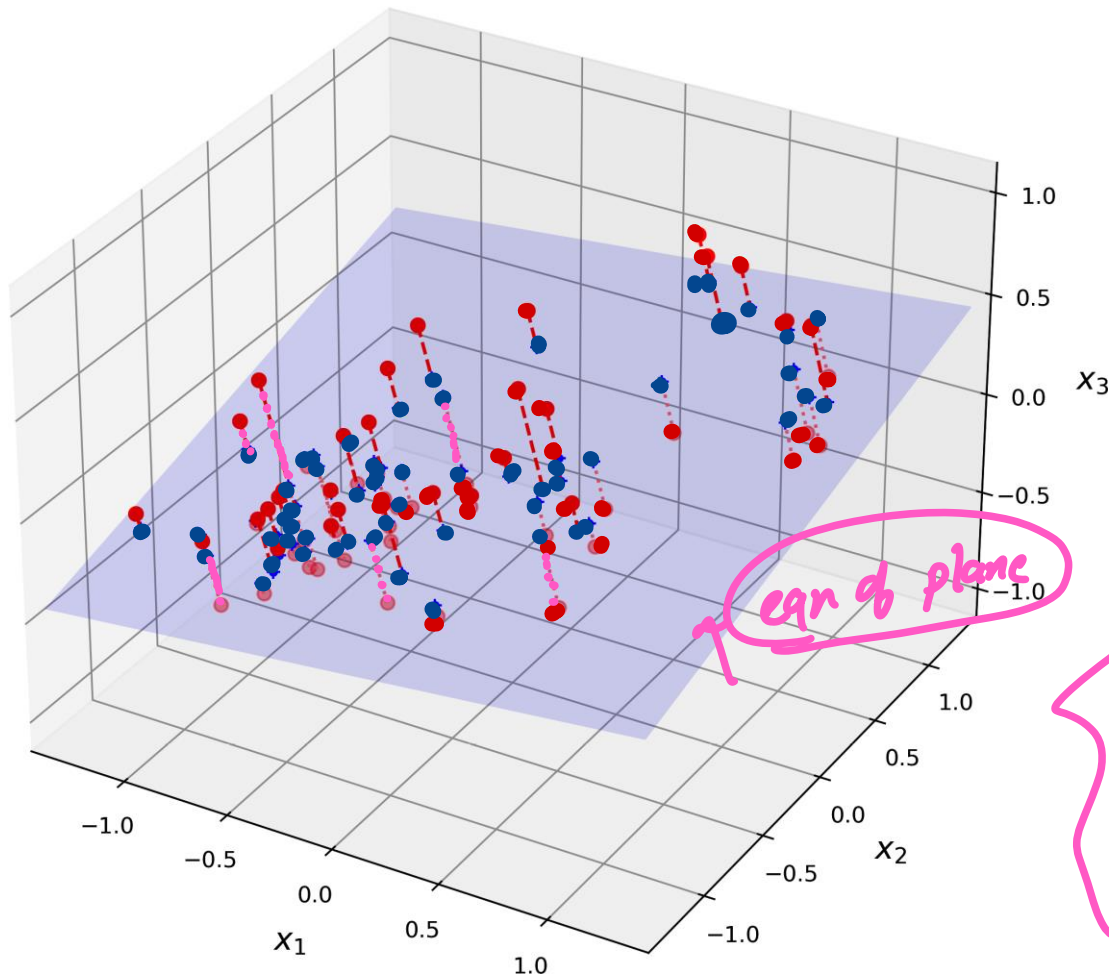
$$A \hat{f}_i = \lambda_i \hat{f}_i$$

Curse of Dimensionality



Projection:

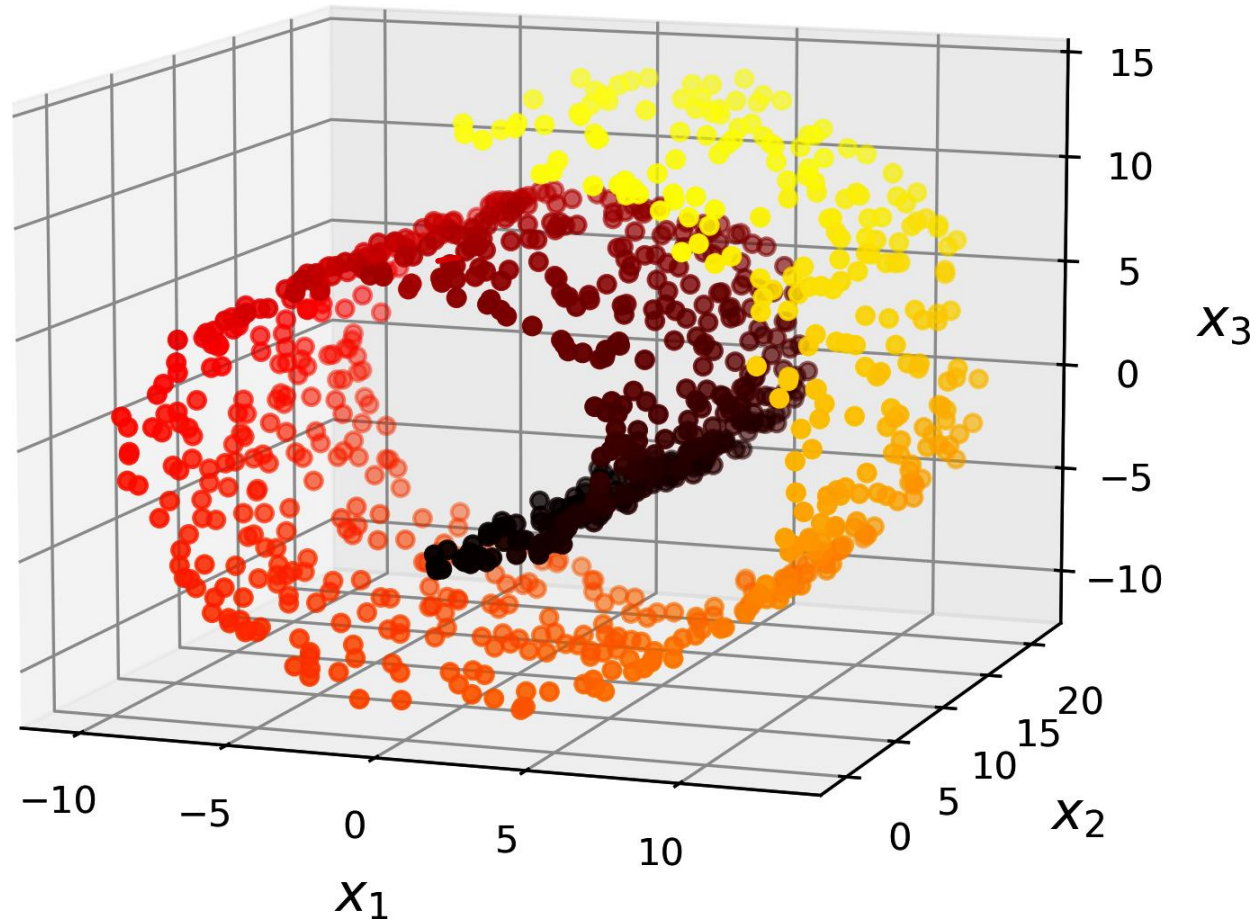
A 3D dataset lying close to a 2D subspace



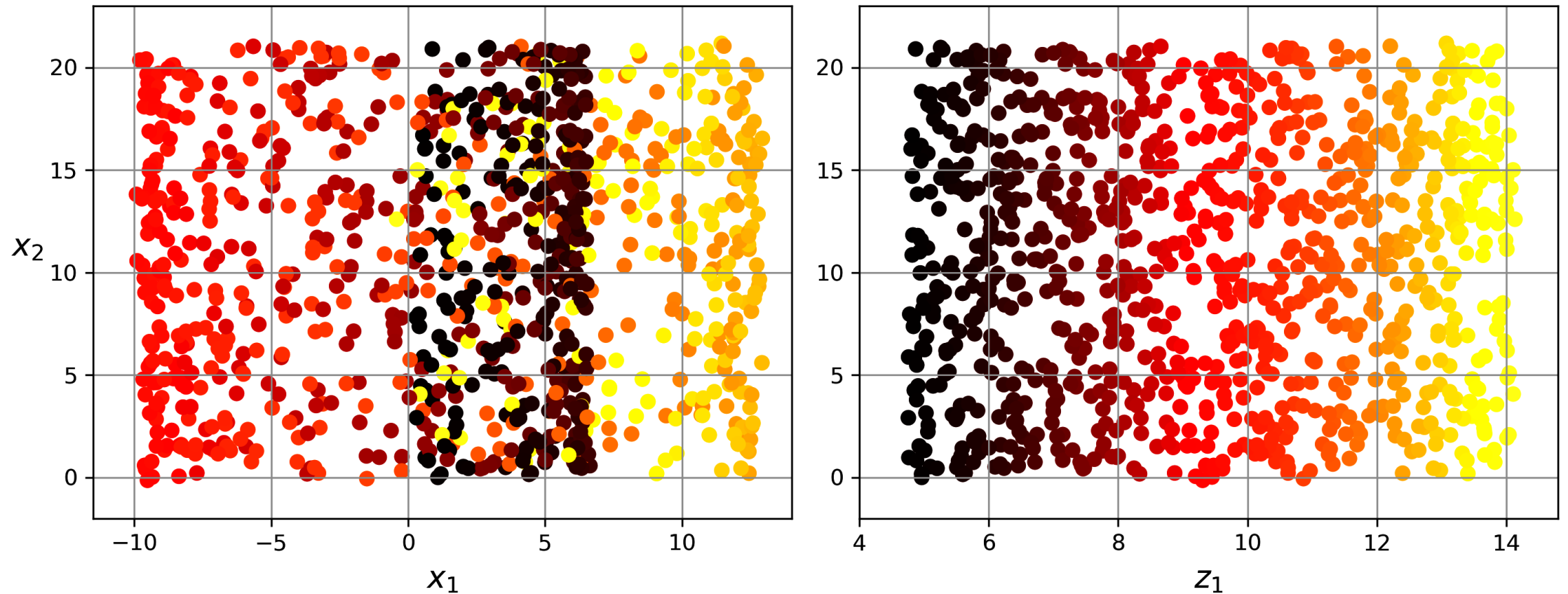
Is Projection a v. good method?
for reducing dimensionality?

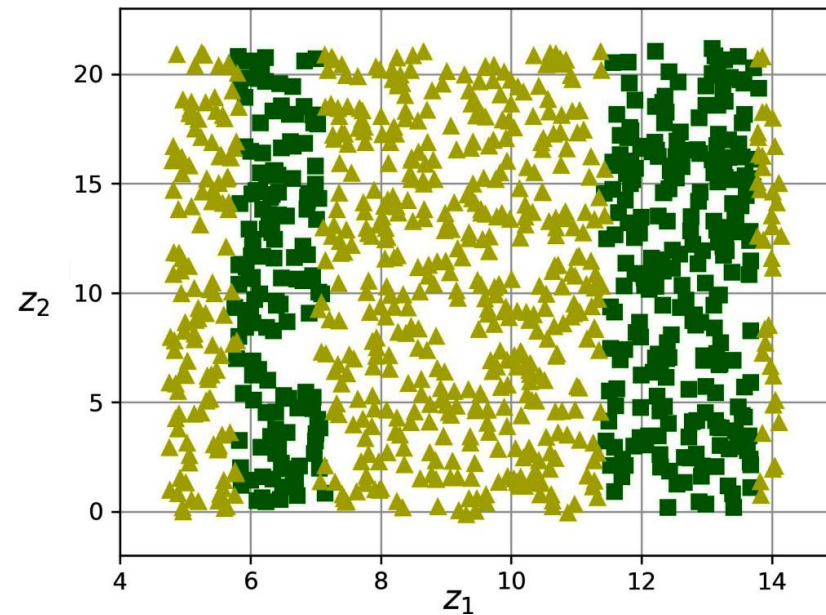
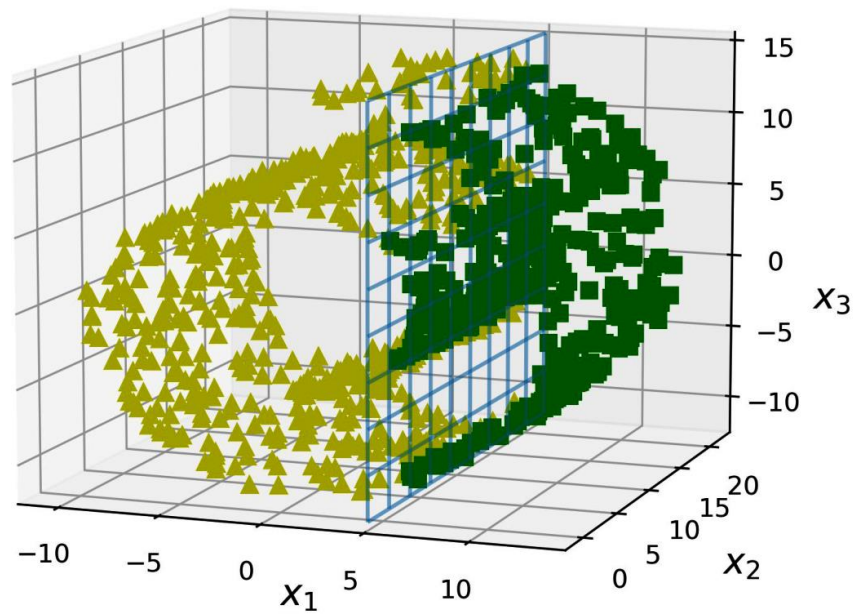
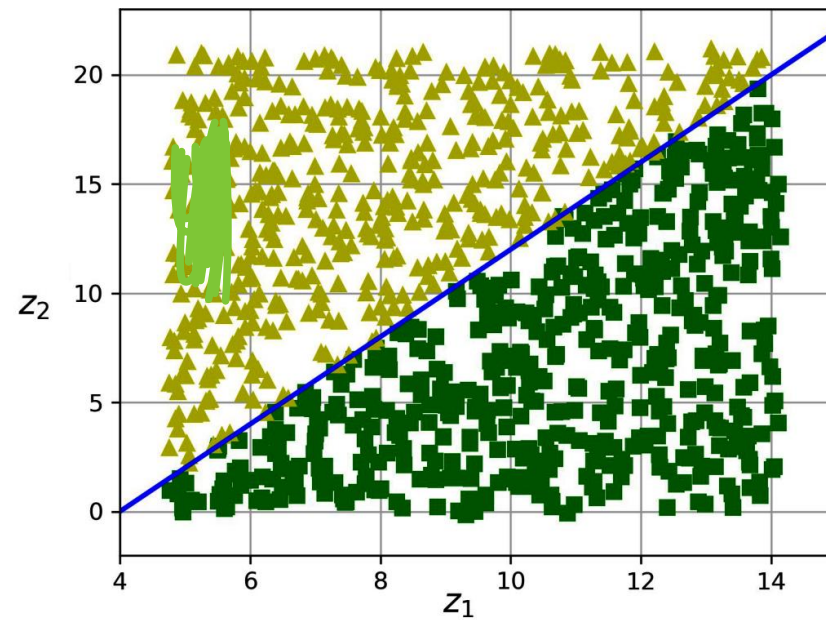
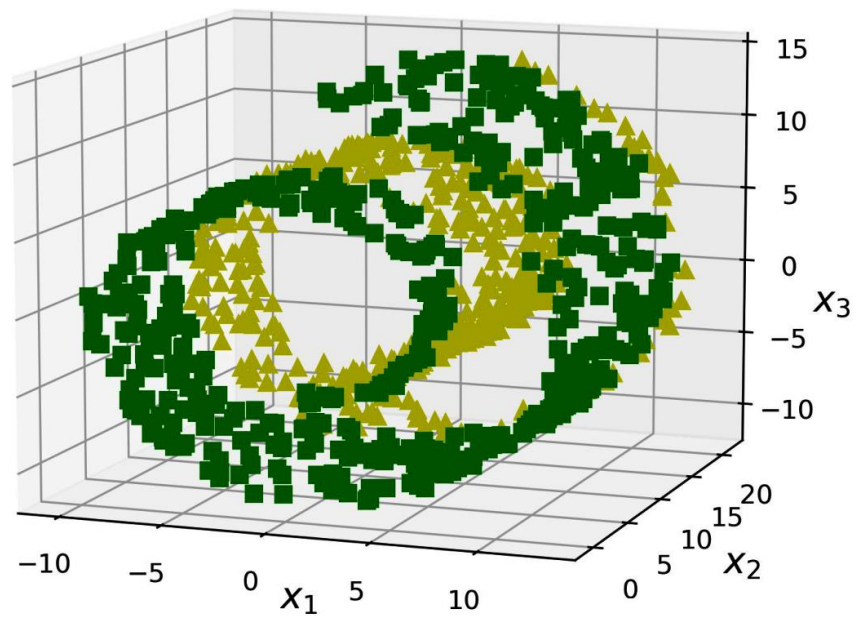
Yes! / Sometimes!

Manifold Learning: Swiss Roll Dataset



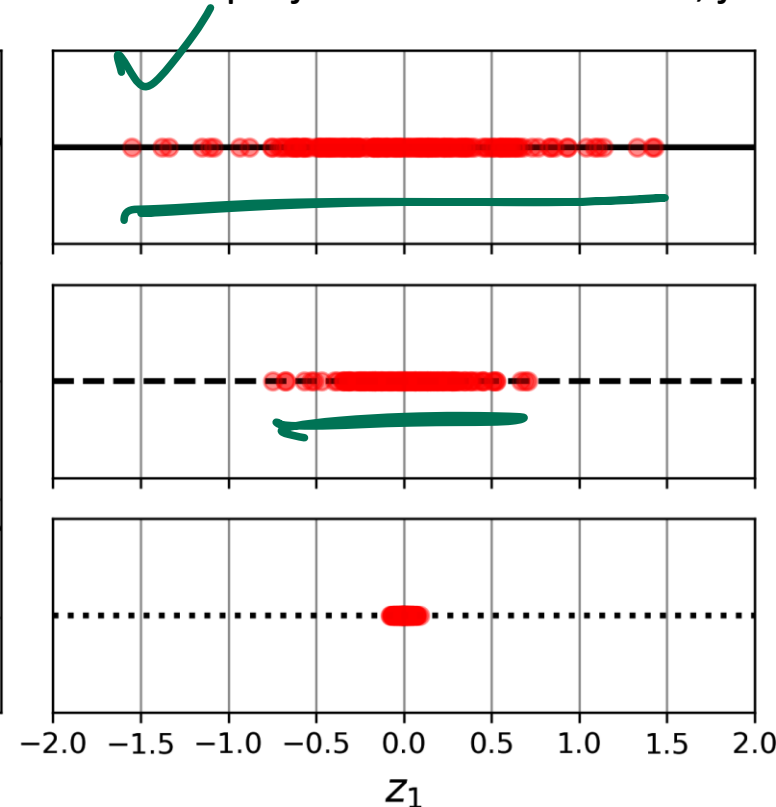
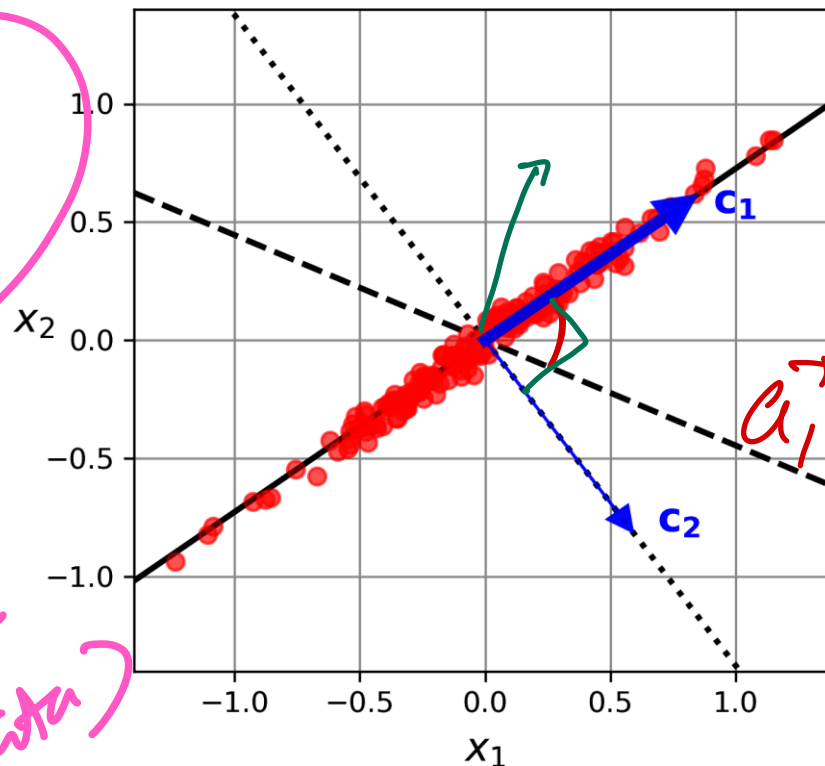
Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right)





Principal Component Analysis

- *Principal component analysis (PCA)* is by far the most popular dimensionality reduction algorithm. First it identifies the hyperplane that lies closest to the data, and then it projects the data onto it, just like in



c_1

$\vec{a}_1$

$\vec{c}_2$

PCA vectors are orthogonal.

(highest variance in data)

PCA

- Preserving the Variance

- Before you can project the training set onto a lower-dimensional hyperplane, you first need to choose the right hyperplane. For example, a simple 2D dataset is represented on the left in the above figure, along with three different axes (i.e., 1D hyperplanes). On the right is the result of the projection of the dataset onto each of these axes.
- As you can see, the projection onto the solid line preserves the maximum variance (top), while the projection onto the dotted line preserves very little variance (bottom) and the projection onto the dashed line preserves an intermediate amount of variance (middle).
- It seems reasonable to select the axis that preserves the maximum amount of variance, as it will most likely lose less information than the other projections. Another way to justify this choice is that it is the axis that minimizes the mean squared distance between the original dataset and its projection onto that axis. This is the rather simple idea behind PCA.

PCA

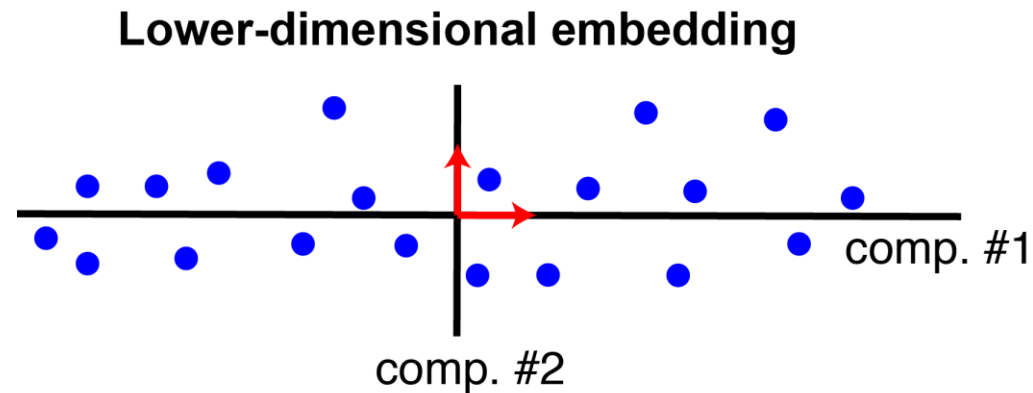
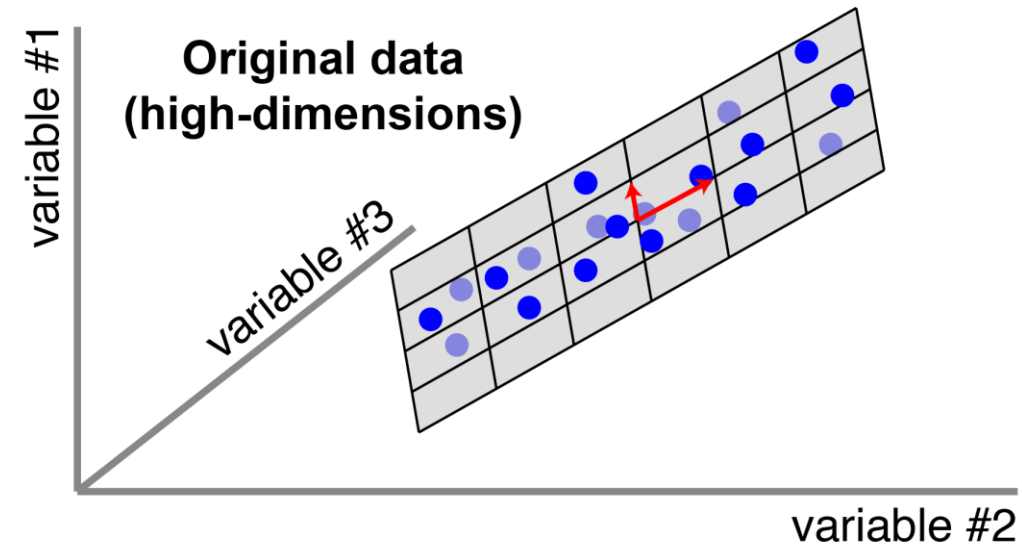
- PCA identifies the axis that accounts for the largest amount of variance in the training set.
- It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of the remaining variance. In this 2D example there is no choice: it is the dotted line.
- If it were a higher-dimensional dataset, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on—as many axes as the number of dimensions in the dataset.
- How to find the PCs of a training set ... Use the **SVD**.

```
import numpy as np
```

```
X = [...] # create a small 3D dataset  
X_centered = X - X.mean(axis=0)  
U, s, Vt = np.linalg.svd(X_centered)  
c1 = Vt[0]  
c2 = Vt[1]
```

So how can you find the principal components of a training set? Luckily, there is a standard matrix factorization technique called *singular value decomposition* (SVD) that can decompose the training set matrix X into the matrix multiplication of three matrices $U \Sigma V^T$, where V contains the unit vectors that define all the principal components

Principal Component Analysis



Singular Value Decomposition

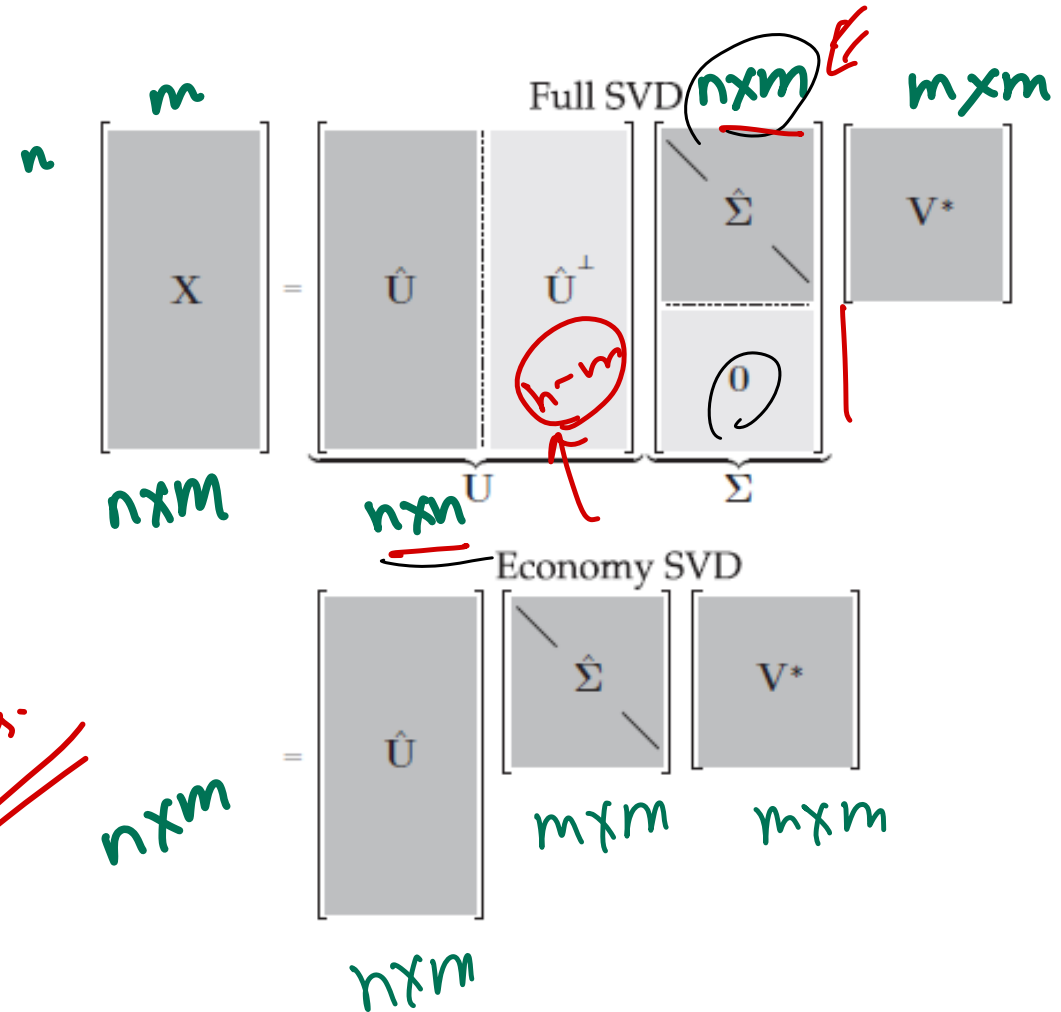
$m \Rightarrow$ examples
 $n \Rightarrow$ length (features)

$n > m$
 $m < n$

$$X = U \Sigma V^T$$

most imp vectors /
building blocks

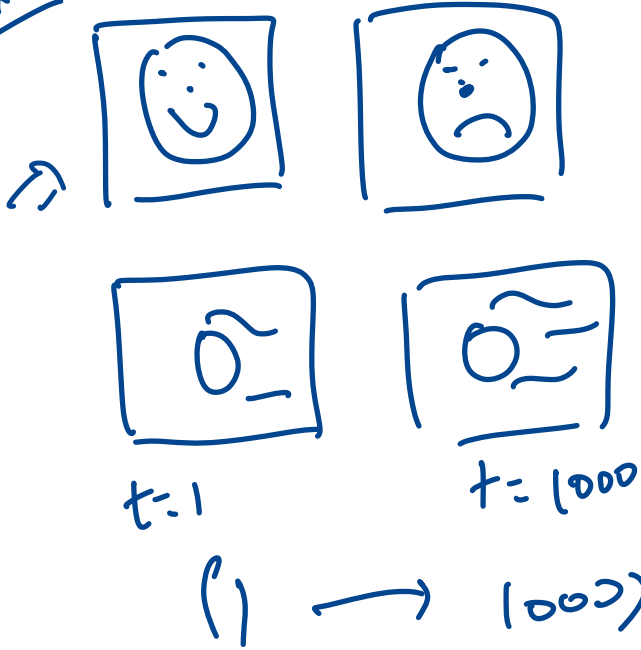
$m \times n$



$$\underline{\underline{X}} = \begin{bmatrix} \vdots & \vdots & \dots & \vdots \\ x_1 & x_2 & \dots & x_m \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

m examples
n length of that matrix
 $x \in \mathbb{R}^n$

n x m



Building blocks

$$\underline{\underline{X}} = U \Sigma V^T \Rightarrow \text{mixture of } v$$

(faces matrix)

(eigen face)

hierarchy scaling

$$X = U \Sigma V^T$$

U → left singular vector
 V^T → right singular vector
 Σ → singular values
 Assume $n > m$

$$= \begin{bmatrix} \underline{\underline{\dot{v}_1}} & \dots & \dot{v}_n \\ \vdots & & \vdots \end{bmatrix} \begin{bmatrix} \sigma_1 & & 0 \\ & \ddots & \\ 0 & & \sigma_m \\ \hline 0 & & 0 \end{bmatrix} \begin{bmatrix} v_1 & \dots & v_m \end{bmatrix}^T$$

$n \times n$
 $m \times m$
 $n \times m$
 $m \times m$
 $v_1 \Rightarrow n \times 1$

U vectors give a basis to represent X and are arranged in hierarchical order of importance

$$\underline{U U^T} = I \quad (n \times n)$$

$$V V^T = I \quad (\underline{m \times m})$$

$$\underline{\Sigma} \Rightarrow \sigma_1 \geq \sigma_2 \dots \geq \sigma_m \geq 0$$

all are non negative.

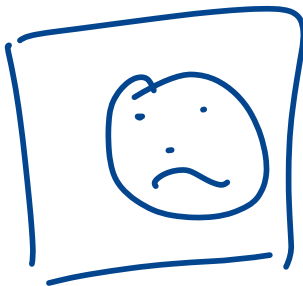
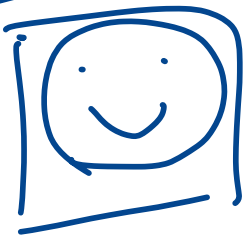
Σ ordered by importance

$$\underline{X} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_m \\ | & | & & | \end{bmatrix}$$

$$\underline{X} = U \overset{\text{diag.}}{\Sigma} V^T$$

$\downarrow$ $n \times n$ $\downarrow$ $m \times m$
 $x \in \mathbb{R}^n$

$n \times m$
Eigen faces



$X \Rightarrow$ face matrix (data)

$U \Rightarrow$ eigen faces (basis vectors).

$\Sigma \Rightarrow$ hierarchy $\Rightarrow$ (scaling)

$V \Rightarrow$ mixture of U

$$U \Sigma V^T = \sigma_1 U_1 V_1^T + \sigma_2 U_2 V_2^T + \dots + \sigma_m U_m V_m^T + 0$$

economy SVD

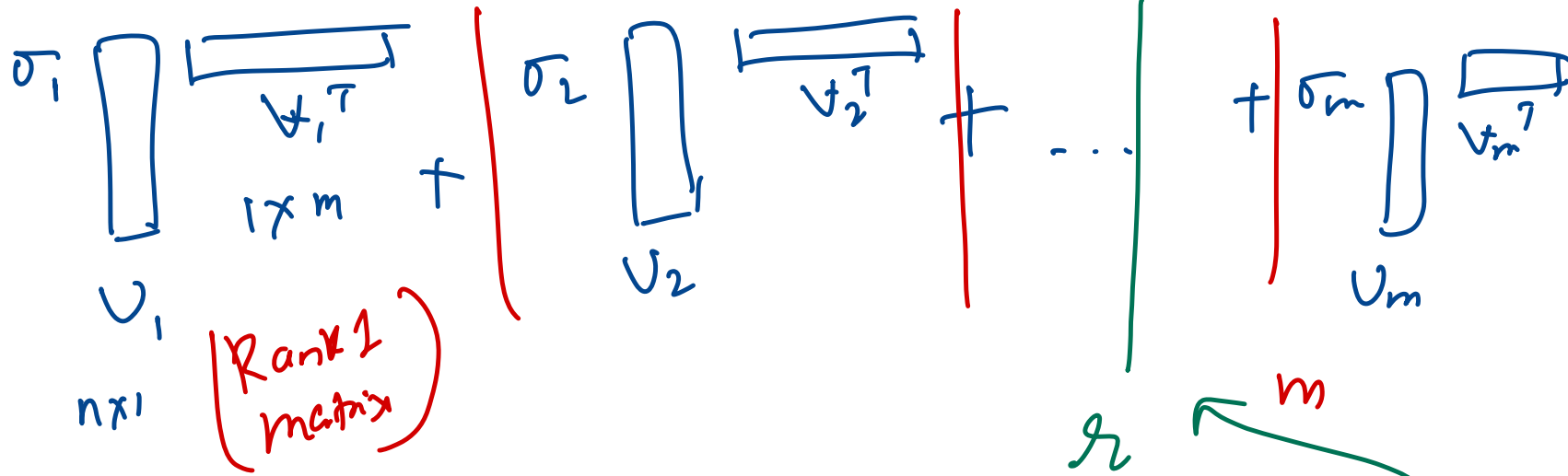
$$U \Sigma V^T = \begin{bmatrix} \end{bmatrix}_{n \times m} \begin{bmatrix} \end{bmatrix}_{m \times m} \begin{bmatrix} \end{bmatrix}_{m \times m}$$

↳
(returns only
the first
m columns.)

$n \gg m$

$$\hat{X} = \hat{U} \hat{\Sigma} \hat{V}^T$$

↳ only for first m values



We don't need a rank m matrix $r \ll m$ to best represent data

Keep the first r columns of U or only r matrices

$$(\tilde{U} \tilde{\Sigma} \tilde{V}^T) \text{ rank } r$$

(econ-SVD) $_r$

$$U \Rightarrow n \times n \quad (\text{full svd})$$

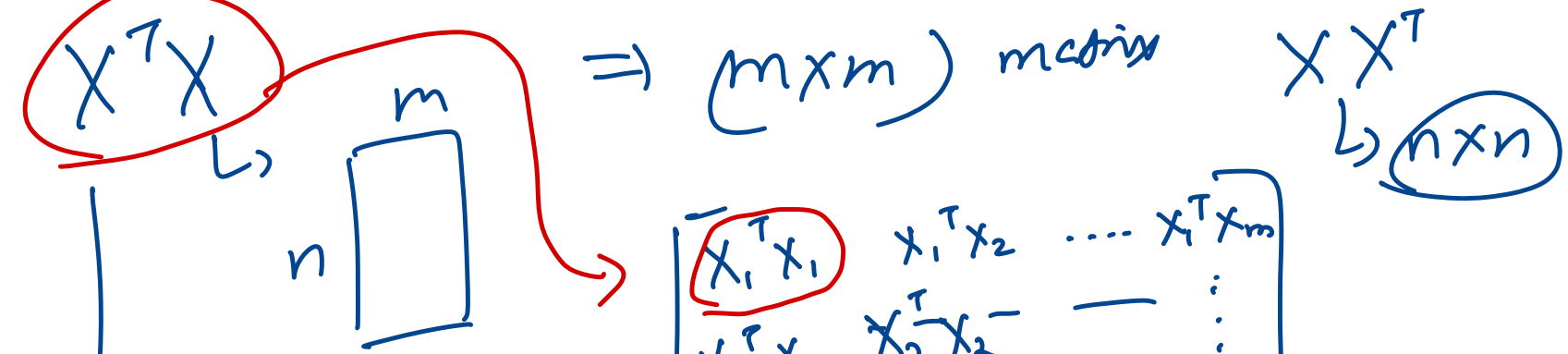
$$U \Rightarrow n \times m \quad (\text{econ-svd})$$

$$U \Rightarrow n \times k \quad (\text{econ-svd})_k$$

$$\Sigma \Rightarrow n \times m \quad (\text{full svd}) \Rightarrow \text{all rows beyond } m \text{ were } 0$$

$$\Sigma \Rightarrow m \times m \quad (\text{econ-svd})$$

$$\Sigma \Rightarrow k \times m \quad (\text{econ-svd})_k$$



$$\begin{bmatrix}
 X_1^T X_1 & X_1^T X_2 & \dots & X_1^T X_m \\
 X_2^T X_1 & X_2^T X_2 & \dots & \vdots \\
 \vdots & \vdots & \ddots & \vdots \\
 X_m^T X_1 & \dots & \dots & X_m^T X_m
 \end{bmatrix}$$

$X_i^T X_j = \langle X_i X_j \rangle$

Correlation matrix

$$X = \hat{U} \hat{\Sigma} V^T \quad X^T = V \hat{\Sigma} \hat{U}^T \quad \dots \text{econ-SVD}$$

$$X^T X = V \hat{\Sigma} \boxed{\hat{U}^T \hat{U}} \hat{\Sigma} V^T$$

$$A = \Phi \Lambda \Phi^T$$

$$\underline{X^T X} = V \hat{\Sigma}^2 V^T$$

eigen
vector

diagonal

eigen value decomposition of
correlation matrix

$$X^T X V = V \hat{\Sigma}^2$$

$$\boxed{A \vec{v} = \lambda \vec{v}}$$

$$XX^T = \hat{U} \hat{\Sigma} V^T V \hat{\Sigma} U^T$$

$$XX^T = \hat{U} \hat{\Sigma}^2 U^T$$

$\downarrow$ ↪ (eigen value)
 eigen
 vec

$n \times n$ matrix

row wise correlation

$\hat{U}$ & V matrices comprise of eigen vectors
 of XX^T & $X^T X$ corr.
 matrices.

PCA

① $X = \begin{bmatrix} -X_1 & \dots \\ -X_m & \dots \end{bmatrix}$ m examples
n features

② compute mean $m \times n$

$$\bar{x}_{ij} = \frac{1}{n} \sum_{i=1}^n X_{ij} \quad \Bigg| \quad \bar{X} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} [-\bar{x}_1 -]$$

③ subtract the mean

$$B = \underline{X - \bar{X}}$$

$m \times n$

④ compute correlation of B

$$C = B^T B$$

⑤ Compute eigen values of C

$$CV = V \underbrace{D}_{\substack{\text{eigen values} \\ \downarrow \\ \text{eigen vector}}}$$

$V \Rightarrow$ contains eigen vectors.

⑥ $(T = BV)$

(matrix T contains the principal components)

$$T = BV$$

$$B = \underline{\underline{U \Sigma V^T}}$$

$$\therefore T = U \Sigma$$

⑦

$\Rightarrow$ we are doing SVD for mean subtracted data (B)

$\sum$ gives an indication of the variance that PCs capture.

$$\lambda = \sigma^2$$

How much variance is captured by the first k modes

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^m \lambda_i}$$

SVD

- Example of Image Compression
- Data Driven Science and Engineering – Machine Learning, Dynamical Systems, and Control. Steven Brunton, Nathan Kutz.

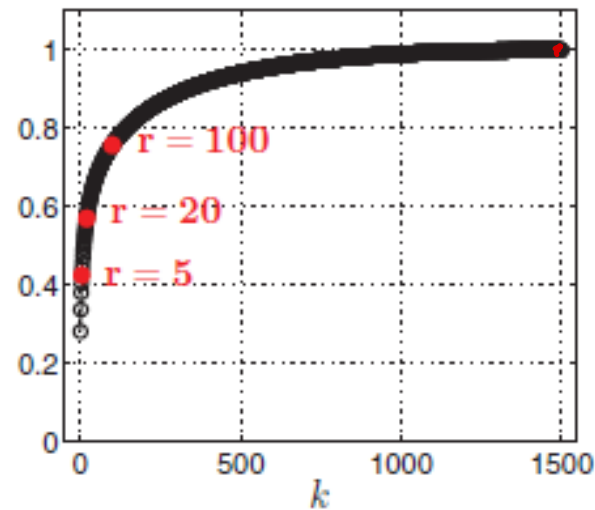
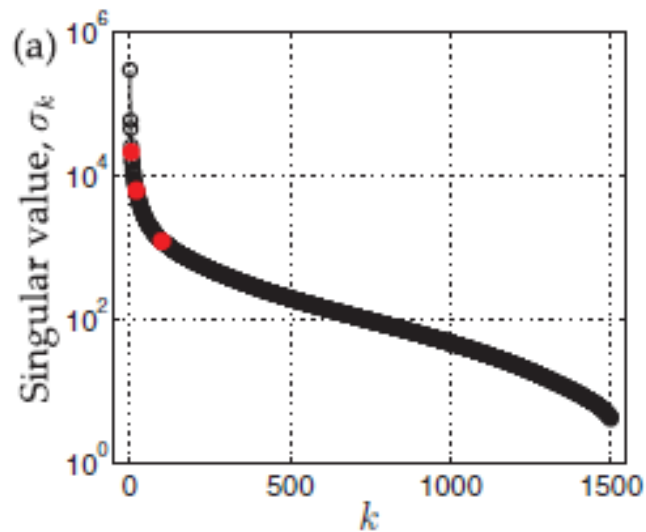


Figure 1.4 (a) Singular values σ_k . (b) Cumulative energy in the first k modes.

Original



$r = 5$, 0.57% storage



$r = 20$, 2.33% storage



$r = 100$, 11.67% storage



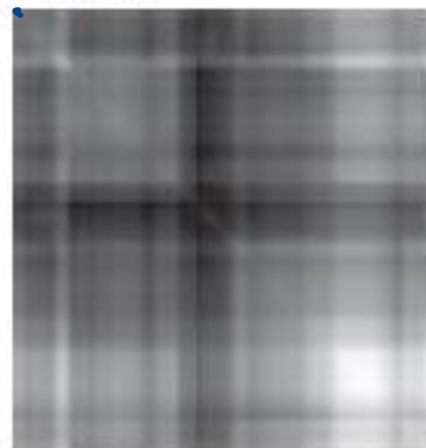
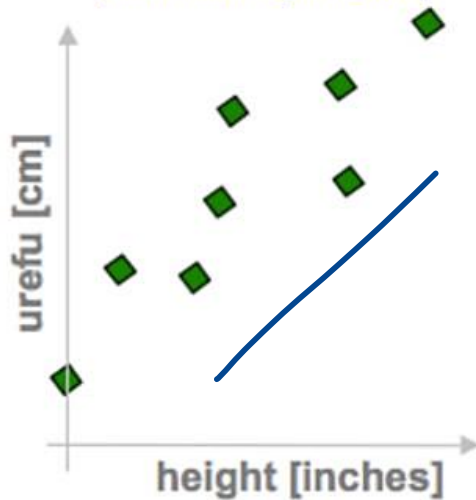
X  XX^*  X^*X 

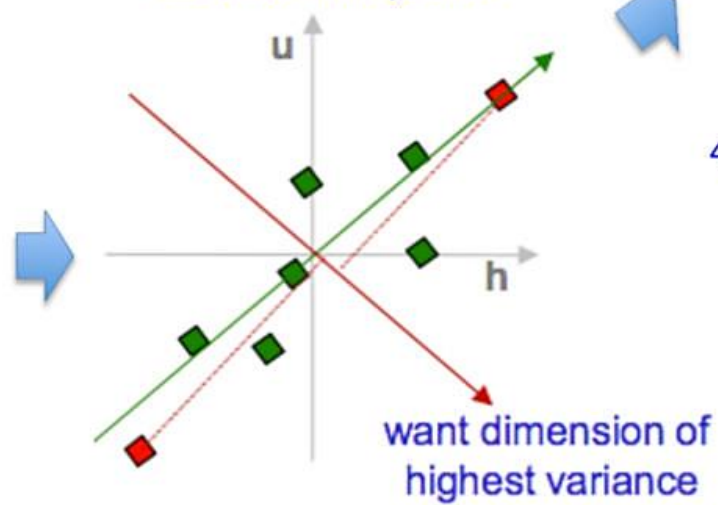
Figure 1.5 Correlation matrices XX^* and X^*X for a matrix X obtained from an image of a dog. Note that both correlation matrices are symmetric.

PCA in a nutshell

1. correlated hi-d data
("urefu" means "height" in Swahili)



2. center the points



3. compute covariance matrix

$$\begin{matrix} & h & u \\ h & \begin{pmatrix} 2.0 & 0.8 \end{pmatrix} \\ u & \begin{pmatrix} 0.8 & 0.6 \end{pmatrix} \end{matrix} \rightarrow \text{cov}(h,u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

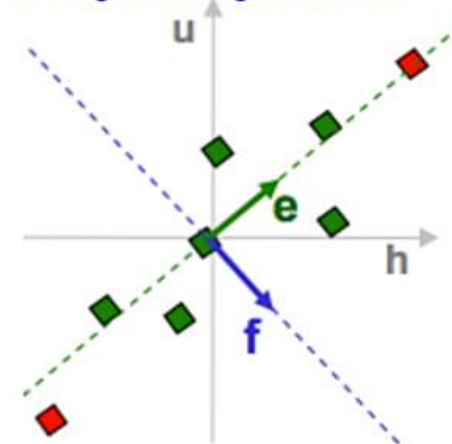
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

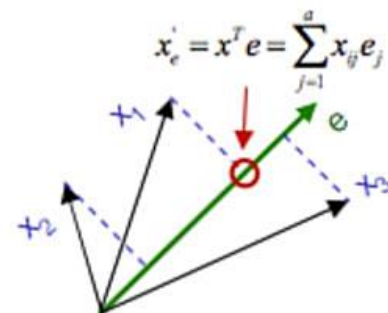
$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

$\text{eig}(\text{cov}(\text{data}))$

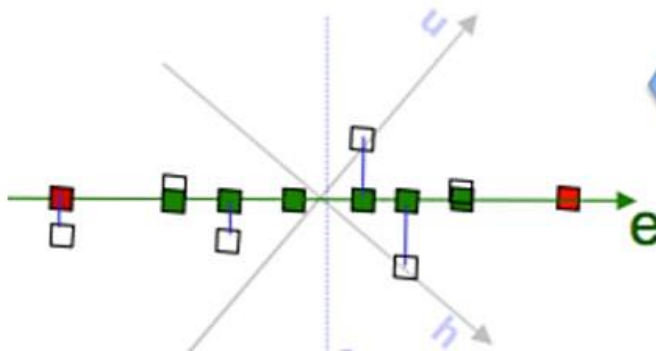
5. pick $m \leq d$ eigenvectors
w. highest eigenvalues



6. project data points to those eigenvectors



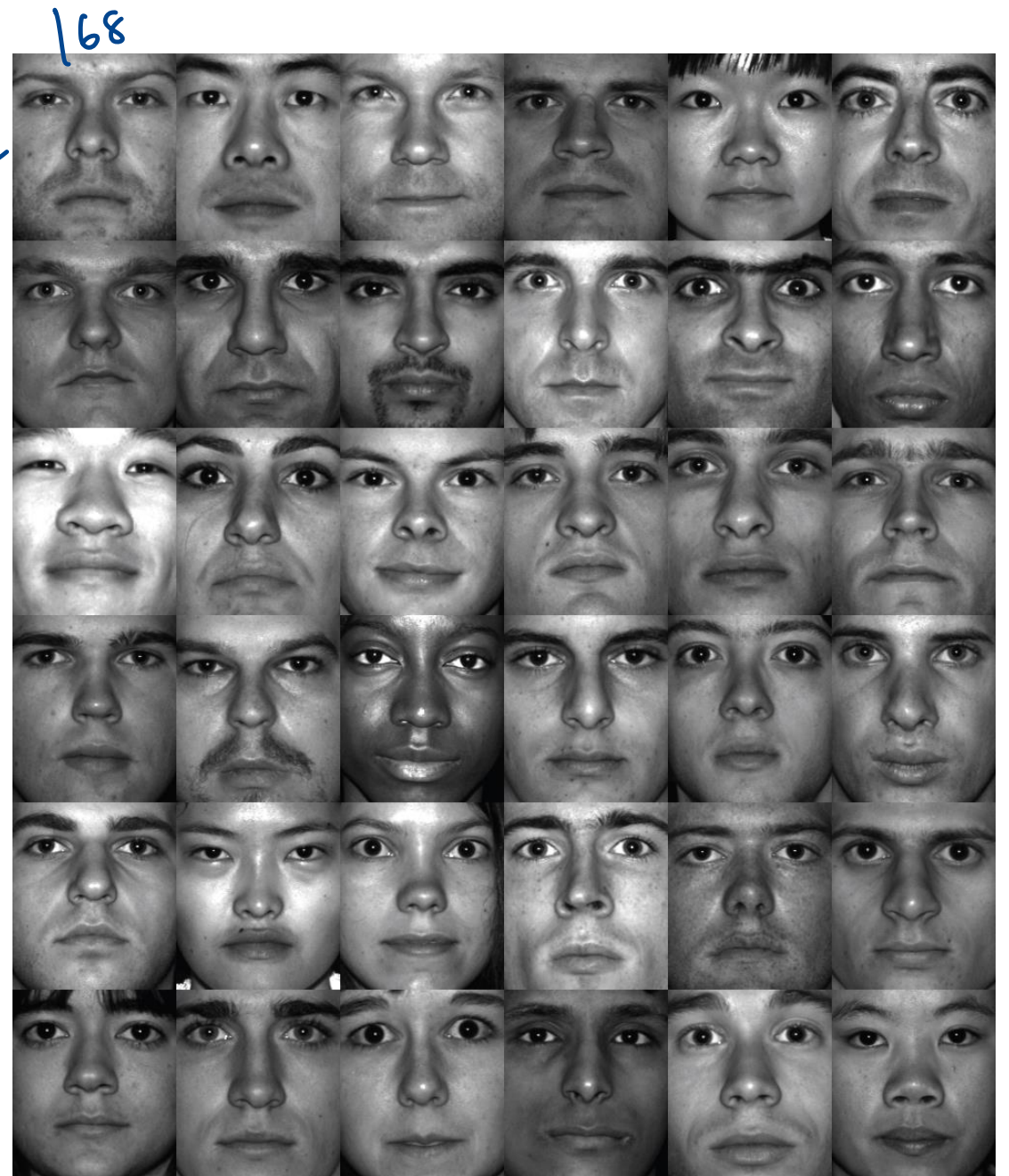
7. uncorrelated low-d data



Eigen Faces Example ¹⁹²

$$\frac{32256}{192 \times 168} \Rightarrow \text{no. of features} \Rightarrow (n)$$

$$m = 36 \times 64 \text{ faces} = 2304$$



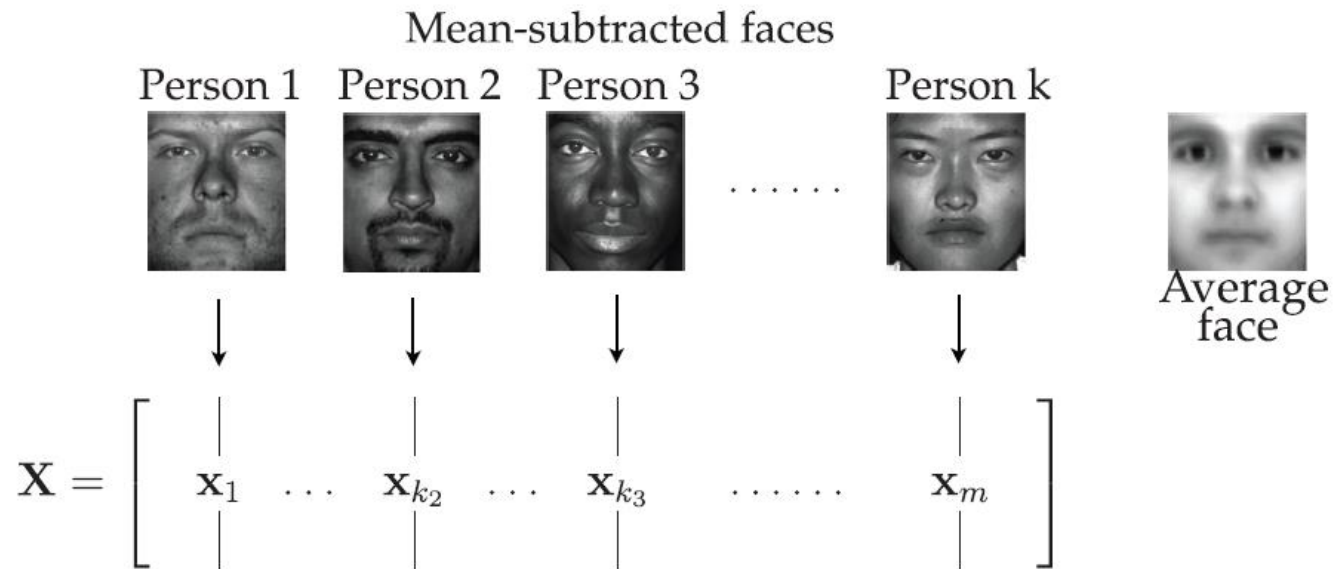
64 faces of
each person



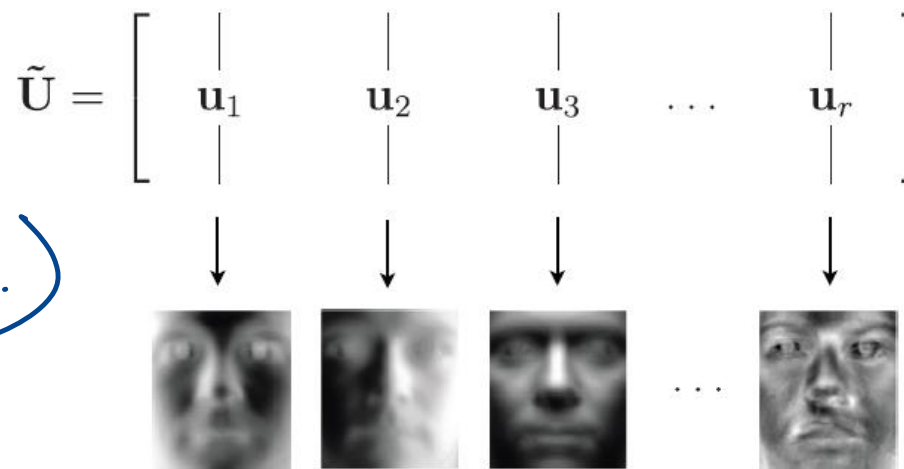
36x64 images
32256 features (pixels)

2304

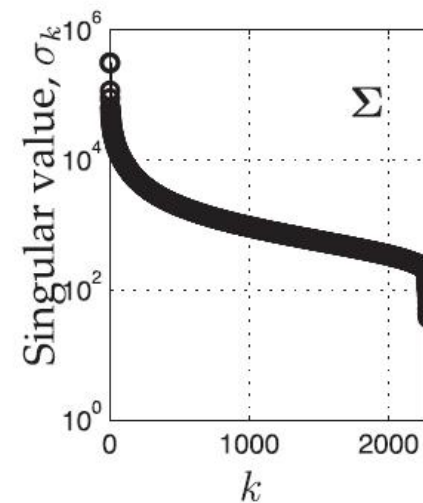
If you do not subtract
 mean face from
 your first eigen
 actually the
 first eigen
 face is
 mean face.)



$$X = U\Sigma V^* \approx \tilde{U}\tilde{\Sigma}\tilde{V}^* \quad (>> [U, S, V] = \text{svd}(X, 'econ');$$



Eigenfaces



Test image



$r = 25$



$r = 50$



$r = 100$



$r = 200$



$r = 400$



$r = 800$



$r = 1600$





Figure 1.19 Approximate representation of an image of a dog using eigenfaces.

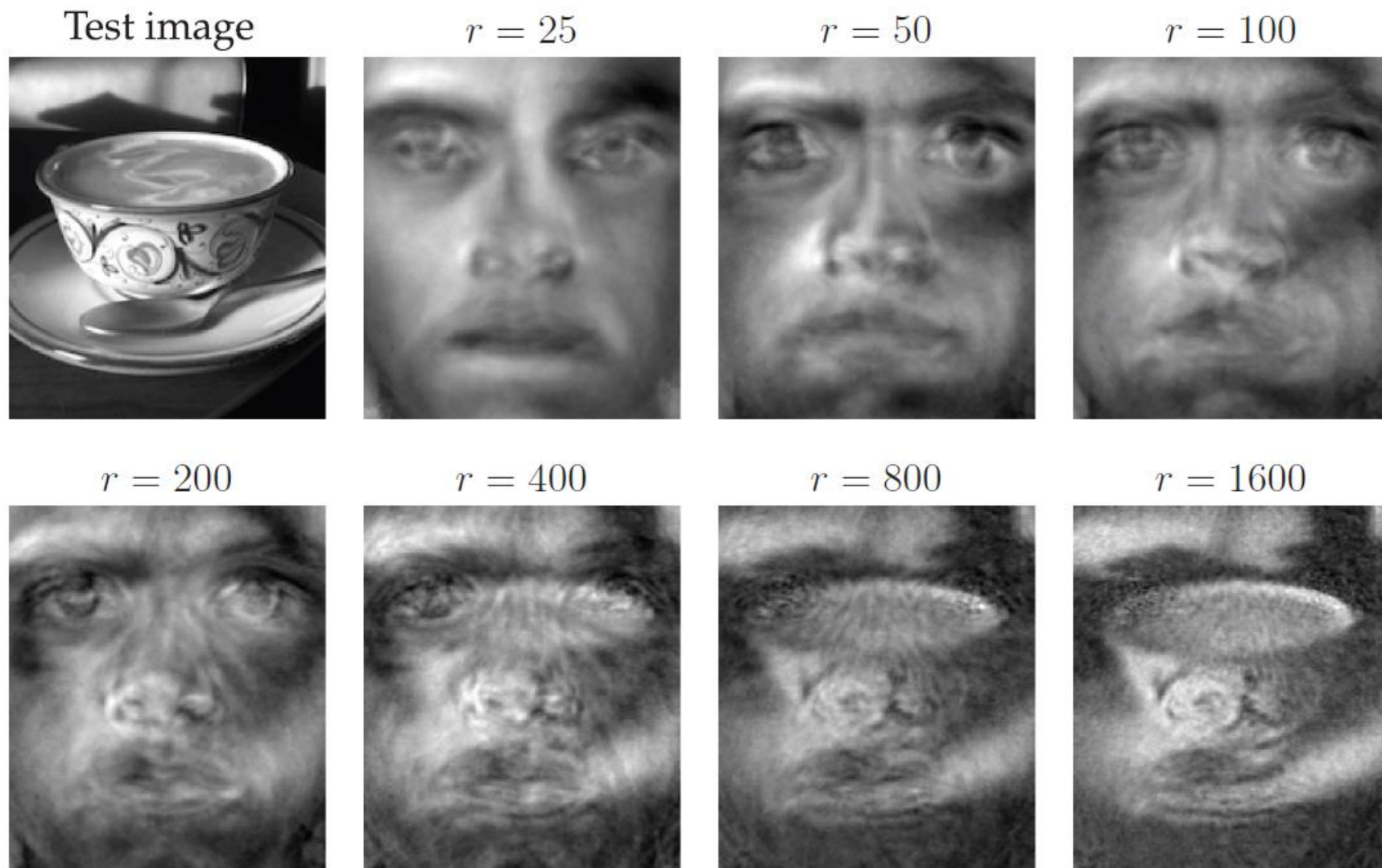
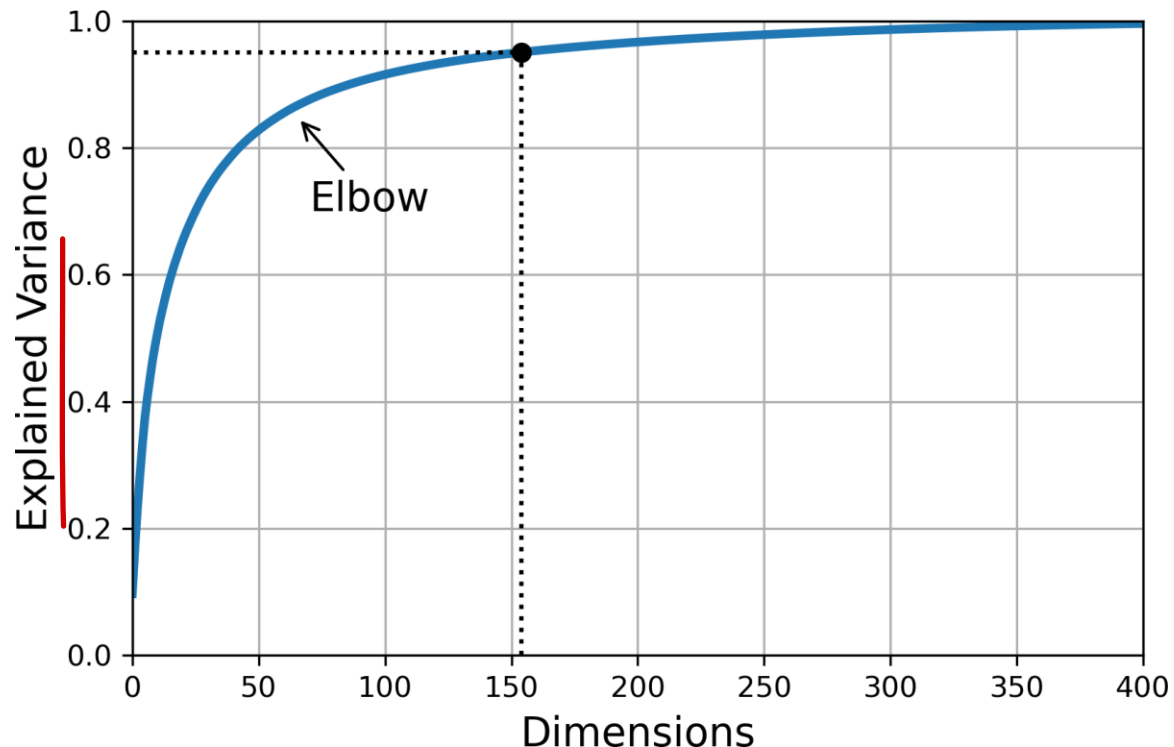


Figure 1.20 Approximate representation of a cappuccino using eigenfaces.

Preserving Variance

Another useful piece of information is the *explained variance ratio* of each principal component, available via the `explained_variance_ratio_` variable. The ratio indicates the proportion of the dataset's variance that lies along each principal component.



```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]

pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1 # d equals 154
```


Self-Reading

- For example, the following code example creates a two-step pipeline, first reducing dimensionality using PCA, then classifying using a random forest. Next, it uses RandomizedSearchCV to find a good combination of hyperparameters for both PCA and the random forest classifier. This example does a quick search, tuning only 2 hyperparameters, training on just 1,000 instances, and running for just 10 iterations, but feel free to do a more thorough search if you have the time:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                               random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

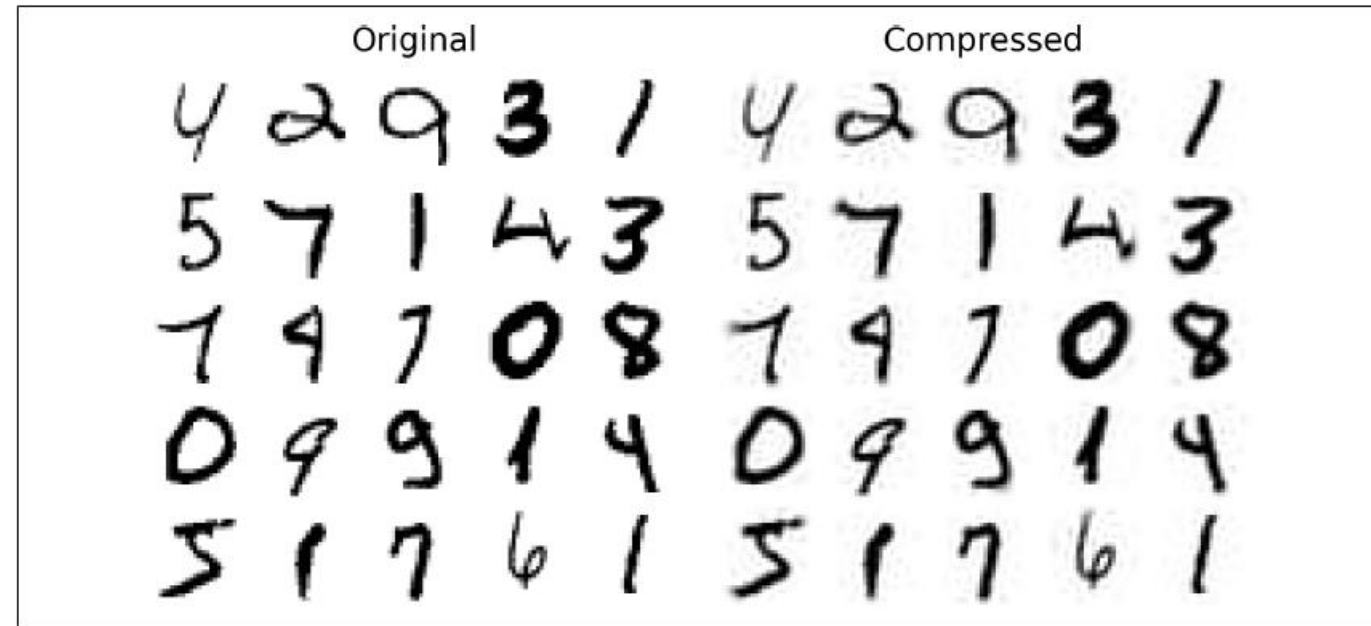
Let's look at the best hyperparameters found:

```
>>> print(rnd_search.best_params_)
{'randomforestclassifier__n_estimators': 465, 'pca__n_components': 23}
```

PCA for Compression

```
X_recovered = pca.inverse_transform(X_reduced)
```

- MNIST Data Set
- Reconstruction Error



Equation 8-3. PCA inverse transformation, back to the original number of dimensions

$$\mathbf{X}_{\text{recovered}} = \mathbf{X}_{d\text{-proj}} \mathbf{W}_d^T$$

Randomized PCA

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)  
X_reduced = rnd_pca.fit_transform(X_train)
```

- Stochastic algorithm called *randomized PCA* that quickly finds an approximation of the first d principal components.

Incremental PCA

Allows data to be split up into mini-batches at a given time.

Good for large data sets.

However, for data sets with very large features:
PCA is still slow.

```
from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

Random Projection

- The random projection algorithm projects the data to a lower dimensional space using a random linear projection.
- Johnson and Lindenstrauss came up with an equation that determines the minimum number of dimensions to preserve in order to ensure—with high probability—that distances won't change by more than a given tolerance.
- For example, if you have a dataset containing $m = 5,000$ instances with $n = 20,000$ features each, and you don't want the squared distance between any two instances to change by more than $\epsilon = 10\%$.
- One should project the data down to d dimensions, with $d \geq 4 \log(m) / (1/2 \epsilon^2 - 1/3 \epsilon^3)$, which is 7,300 dimensions
- Notice that the equation does not use n , it only relies on m and ϵ .

```
>>> from sklearn.random_projection import johnson_lindenstrauss_min_dim
>>> m, ε = 5_000, 0.1
>>> d = johnson_lindenstrauss_min_dim(m, eps=ε)
>>> d
7300
```

Now we can just generate a random matrix $\mathbf{P}$ of shape $[d, n]$, where each item is sampled randomly from a Gaussian distribution with mean 0 and variance $1 / d$, and use it to project a dataset from n dimensions down to d :

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d)  # std dev = square root of variance

X = np.random.randn(m, n)  # generate a fake dataset
X_reduced = X @ P.T
```

That's all there is to it! It's simple and efficient, and no training is required: the only thing the algorithm needs to create the random matrix is the dataset's shape. The data itself is not used at all.

Scikit-Learn offers a `GaussianRandomProjection` class to do exactly what we just did: when you call its `fit()` method, it uses `johnson_lindenstrauss_min_dim()` to

Local Linear Embedding

- Non-linear dimensionality reduction technique
- Manifold learning technique that does not rely on projections unlike PCA.
- LLE works by first measuring how each training instance linearly relates to its nearest neighbors, and then looking for a low-dimensional representation of the training set where these local relationships are best preserved.

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```


Here's how LLE works: for each training instance $\mathbf{x}^{(i)}$, the algorithm identifies its k -nearest neighbors (in the preceding code $k = 10$), then tries to reconstruct $\mathbf{x}^{(i)}$ as a linear function of these neighbors. More specifically, it tries to find the weights $w_{i,j}$ such that the squared distance between $\mathbf{x}^{(i)}$ and $\sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)}$ is as small as possible, assuming $w_{i,j} = 0$ if $\mathbf{x}^{(j)}$ is not one of the k -nearest neighbors of $\mathbf{x}^{(i)}$. Thus the first step of LLE is the constrained optimization problem described in Equation 8-4, where $\mathbf{W}$ is the weight matrix containing all the weights $w_{i,j}$. The second constraint simply normalizes the weights for each training instance $\mathbf{x}^{(i)}$.

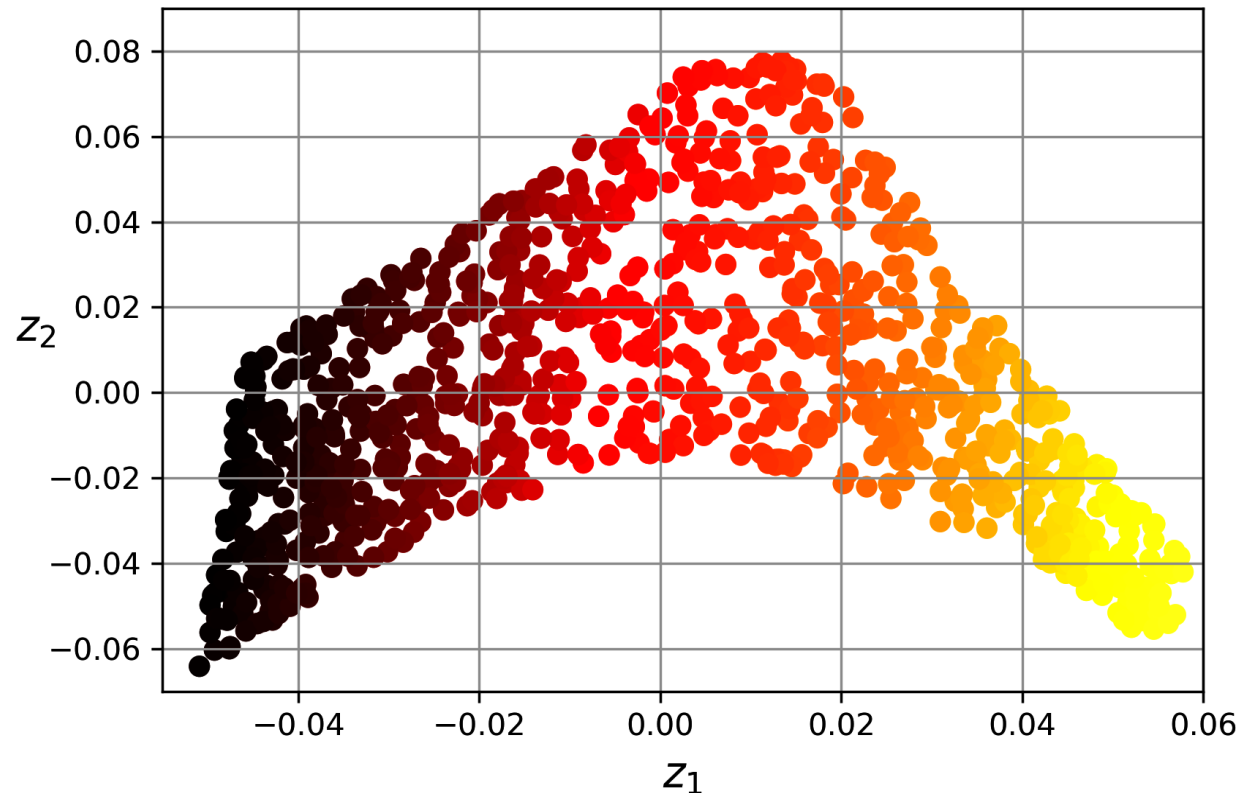
Equation 8-4. LLE step 1: linearly modeling local relationships

$$\begin{aligned} \widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \quad & \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \\ \text{subject to} \quad & \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ n.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases} \end{aligned}$$

- Refer to Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow by Geron.

Equation 8-5. LLE step 2: reducing dimensionality while preserving relationships

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$



Other Dimensionality Reduction Techniques

`sklearn.manifold.MDS`

Multidimensional scaling (MDS) reduces dimensionality while trying to preserve the distances between the instances. Random projection does that for high-dimensional data, but it doesn't work well on low-dimensional data.

`sklearn.manifold.Isomap`

Isomap creates a graph by connecting each instance to its nearest neighbors, then reduces dimensionality while trying to preserve the *geodesic distances* between the instances. The geodesic distance between two nodes in a graph is the number of nodes on the shortest path between these nodes.

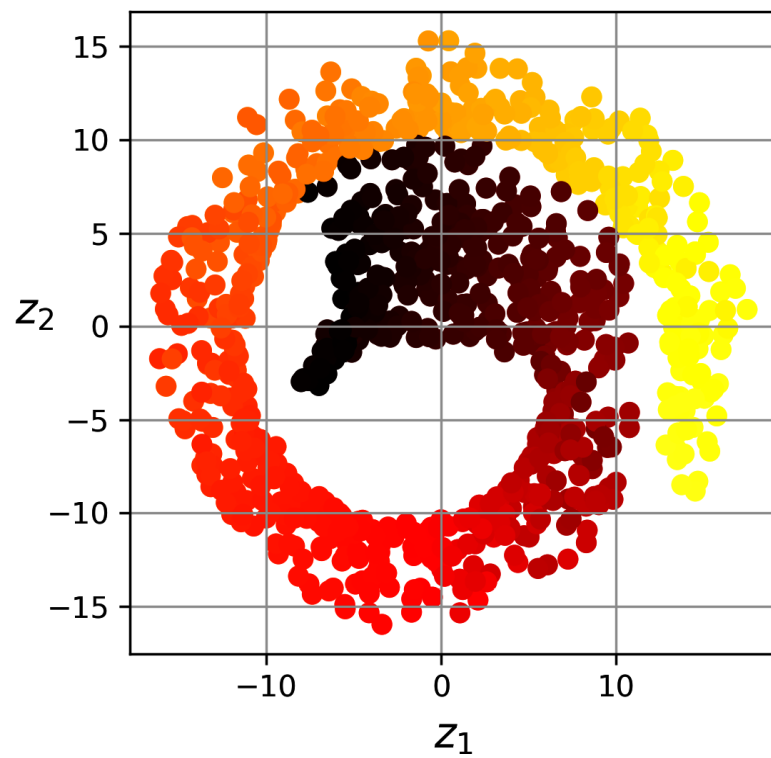
`sklearn.manifold.TSNE`

t-distributed stochastic neighbor embedding (t-SNE) reduces dimensionality while trying to keep similar instances close and dissimilar instances apart. It is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space. For example, in the exercises at the end of this chapter you will use t-SNE to visualize a 2D map of the MNIST images.

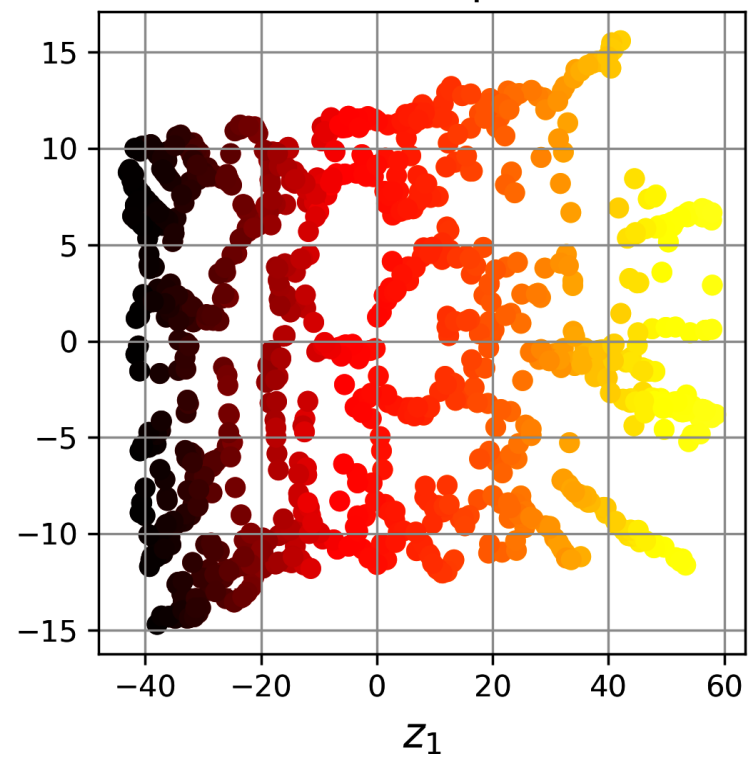
`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

Linear discriminant analysis (LDA) is a linear classification algorithm that, during training, learns the most discriminative axes between the classes. These axes can then be used to define a hyperplane onto which to project the data. The benefit of this approach is that the projection will keep classes as far apart as possible, so LDA is a good technique to reduce dimensionality before running another classification algorithm (unless LDA alone is sufficient).

MDS



Isomap



t-SNE

